C++14 Fundamentals - Student Edition

Week 2 - Class 4: Floating Point Basics

Why This Matters

Integers are not enough to represent real-world values like 3.14159 or 98.6. In Python, float hides the details. In C++, you must choose between float, double, and long double.

Floating Point Types

- float: 32 bits, ~7 digits precision.
- double: 64 bits, ~15-17 digits precision.
- long double: at least as precise as double; may be 80 or 128 bits on some systems, but often the same as double.

Always check sizeof() for your system.

IEEE-754 Representation

Floating point numbers are stored using IEEE-754:
- Sign bit (1)
- Exponent (scales by power of 2)
- Mantissa (fraction)

Value = (-1)^sign * 1.mantissa * 2^(exponent-bias).

Because many decimals cannot be represented in binary, 0.1 and 0.2 are not exact.

Demonstration: 0.1 + 0.2

In Python: 0.1+0.2=0.30000000000000004.
In C++ the same happens with double.

```
#include <iostream>
using namespace std;
int main() {
    double x = 0.1 + 0.2;
    cout << x << "\n"; // prints 0.30000000000000004
}
```

Example Program: Comparing Float vs Double

Compare precision of float, double, long double.

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    float f = 1.0f/3.0f;
    double d = 1.0/3.0;
    long double ld = 1.0L/3.0L;
    cout << setprecision(20);
    cout << "float 1/3 = " << f << "\n";
    cout << "double 1/3 = " << d << "\n";
    cout << "long double 1/3 = " << ld << "\n";
}
```

ASCII Table: Precision and Size

```
+-------------+-----------+-----------------+
| Type        | Size      | Precision (digits)|
+-------------+-----------+-----------------+
| float       | 4 bytes   | ~7              |
| double      | 8 bytes   | ~15             |
```

| long double | 8-16 bytes| 15-18 |
+--------------+-----------+-----------------+

Common Mistakes

1. Assuming decimals are exact. 0.1 is not exact.
2. Comparing floats with ==. Use tolerance.
3. Believing long double always improves precision. Sometimes it equals double.

Exercises

1. Print 1/3 as float,double,long double.
2. Print pi at precision 3,6,9,15.
3. Test 0.1+0.2==0.3.
4. Print difference between (0.1+0.2) and 0.3.

Project Tie-In

Extend your Unit Converter project to include floating-point conversions, e.g. Celsius<->Fahrenheit. Experiment with different precisions.